



Wie verhindere ich in einer Flask-Route mit Flask-Login, dass ein eingeloggter User sich auf seinen eigenen Eintrag bewirbt (Vergleich der User-ID mit der Ersteller-ID)?

Du verhinderst das, indem du in der Route prüfst, ob `current_user.id` mit der `owner_id` (Ersteller-ID) des Eintrags übereinstimmt, und bei Gleichheit eine Fehlermeldung zurückgibst oder umleitest. ^[1] ^[2]

Grundprinzip

Flask-Login stellt dir mit `current_user` ein globales Objekt bereit, das den aktuell eingeloggten User repräsentiert. In einer Route, die über eine 提交的 Bewerbung (z. B. auf einen Job/Post) handelt, lädst du zuerst den Ziel-Eintrag aus der Datenbank und vergleichst dann: ^[1]

- `current_user.id` → ID des eingeloggten Users
- `eintrag.owner_id` (oder ähnlich) → ID des Users, der den Eintrag erstellt hat

Wenn beide IDs gleich sind, soll keine Bewerbung erlaubt werden.

Beispiel-Implementierung

Angenommen, du hast ein Modell `Post` mit einer `owner_id` und eine Route `/post/<int:post_id>/apply`:

```
from flask import Blueprint, abort, flash, redirect, url_for, render_template
from flask_login import login_required, current_user
from yourapp.models import Post, Application

bp = Blueprint('main', __name__)

@bp.route('/post/<int:post_id>/apply', methods=['GET', 'POST'])
@login_required
def apply_to_post(post_id):
    post = Post.query.get_or_404(post_id)

    # Verhindern, dass der Ersteller sich auf den eigenen Eintrag bewirbt
    if post.owner_id == current_user.id:
        flash('Du kannst dich nicht auf deinen eigenen Eintrag bewerben.', 'error')
        return redirect(url_for('main.show_post', post_id=post.id))

    # Optional: prüfen, ob schon eine Bewerbung existiert
    existing = Application.query.filter_by(
```

```

        user_id=current_user.id,
        post_id=post.id
    ).first()
    if existing:
        flash('Du hast dich bereits auf diesen Eintrag beworben.', 'info')
        return redirect(url_for('main.show_post', post_id=post.id))

    if request.method == 'POST':
        # Bewerbung erstellen
        application = Application(user_id=current_user.id, post_id=post.id)
        db.session.add(application)
        db.session.commit()
        flash('Deine Bewerbung wurde erfolgreich abgeschickt.', 'success')
        return redirect(url_for('main.show_post', post_id=post.id))

    return render_template('apply.html', post=post)

```

Wichtige Punkte:

- `@login_required` sorgt dafür, dass nur eingeloggte User in die Route kommen. ^[3]
- `post.owner_id == current_user.id` ist die eigentliche Sperre gegen „Selbstbewerbung“. ^[2]
- Bei Verstoß wird per `flash()` eine Meldung gesetzt und mit `redirect()` weg von der apply-Route geleitet. ^[1]

Alternative: Decorator für „Nicht-eigener-Eintrag“

Für mehrere solche Routen kannst du dir einen eigenen Decorator bauen:

```

from functools import wraps
from flask import abort, flash, redirect, url_for
from flask_login import current_user

def not_owner_of(post_attr_name='post'):
    def decorator(f):
        @wraps(f)
        def wrapped(*args, **kwargs):
            post = kwargs.get(post_attr_name)
            if post is None:
                # Fallback, falls du den Eintrag anders übergibst
                post = kwargs.get('item')
            if post and post.owner_id == current_user.id:
                flash('Diese Aktion ist für den eigenen Eintrag nicht erlaubt.', 'error')
                return redirect(url_for('main.show_post', post_id=post.id))
            return f(*args, **kwargs)
        return wrapped
    return decorator

```

Und dann:

```

@bp.route('/post/<int:post_id>/apply', methods=['GET', 'POST'])
@login_required
@not_owner_of('post')

```

```
def apply_to_post(post_id):
    post = Post.query.get_or_404(post_id)
    # ... Rest wie oben
```

Damit musst du die Logik „kein eigener Eintrag“ nicht in jeder Route wiederholen.

Typische Fehlerquellen

- Du vergleichst `不同类型` (z. B. `int` vs. `str`). Stelle sicher, dass `owner_id` und `current_user.id` den gleichen Typ haben.
- Du prüfst zu spät (z. B. erst beim Speichern der Bewerbung). Die Prüfung sollte möglichst früh in der Route passieren, idealerweise direkt nach dem Laden des `Post`-Objekts.
- Du vergisst, bei Verstoß die weitere Verarbeitung abubrechen (kein `return` nach `flash()`).

Wenn du dein konkretes Routing- und Modell-Setup (z. B. Name der Modelle, Routen-Pfad) zeigst, kann ich dir eine passgenaue Variante schreiben.

✱

1. <https://stackoverflow.com/questions/64238325/authentication-url-routing-with-flask-login>
2. <https://www.codementor.io/@olawalealadeusi896/building-a-restful-blog-apis-using-python-and-flask-part-2-l9y8awu5p>
3. <https://stackoverflow.com/questions/57434775/how-to-integrate-login-required-decorator-to-flask-routes-with-a-static-user-id>
4. <https://www.geeksforgeeks.org/python/flask-role-based-access-control/>
5. <https://github.com/learn-co-curriculum/flask-iam-putting-it-all-together-technical-lesson>
6. <https://blog.miguelgrinberg.com/post/the-flask-mega-tutorial-part-v-user-logins>
7. <https://flask.palletsprojects.com/en/stable/tutorial/blog/>
8. <https://medium.com/@elsamatchole/flask-web-development-user-authentication-733c1eea1ccb>
9. <https://flask-login.readthedocs.io/en/0.4.1/pdf/>
10. <https://stackoverflow.com/questions/65434221/prevent-users-from-directly-accessing-url-and-redirect-to-login-if-not-logged-in>